

Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу

«Операционные системы»

Группа: М8О-216БВ-24

Студент: Шитов А.С.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 24.09.25

Москва, 2025

Постановка задачи

Вариант 5.

Родительский процесс создает дочерний процесс. Первой строчкой пользователь в консоль родительского процесса пишет имя файла, которое будет передано при создании дочернего процесса. Родительский и дочерний процессы должны быть представлены разными программами. Родительский процесс передает команды пользователя через `pipe1`, который связан с стандартным входным потоком дочернего процесса. Дочерний процесс при необходимости передает данные в родительский процесс через `pipe2`. Результаты своей работы дочерний процесс пишет в созданный им файл. Допускается просто открыть файл и писать туда, не перенаправляя стандартный поток вывода.

Пользователь вводит команды вида: «число<newline>». Далее это число передается от родительского процесса в дочерний. Дочерний процесс производит проверку на простоту. Если число составное, то это число записывается в файл. Если число отрицательное или простое, то тогда дочерний и родительский процессы завершаются.

Общий метод и алгоритм решения

Использованные системные вызовы:

- `pid_t fork(void)`; – создает дочерний процесс.
- `int pipe(int* fd)`; – создает однонаправленный канал для межпроцессного взаимодействия.
- `int execl(const char* path, const char* arg, ...)`; – заменяет образ текущей программы, на указанную, принимая аргументы в качестве списка.
- `int dup2(int oldfd, int newfd)`; – создает копию файлового дескриптора `oldfd` в указанном дескрипторе `newfd`.
- `int open(const char* pathname, int flags, mode_t mode)`; – открывает файл по указанному пути с заданными флагами и правами доступа.
- `ssize_t read(int fd, void* buf, size_t count)`; – читает данные из файлового дескриптора в буфер.
- `ssize_t write(int fd, const void* buf, size_t count)`; – записывает данные из буфера в файловый дескриптор.
- `int close(int fd)`; – закрывает файловый дескриптор.
- `pid_t wait(int* status)`; – ожидает изменения состояния указанного дочернего процесса.

Целью лабораторной работы было написание программы, которая создает дочерний процесс, подменяет образ программы. Предварительно создается два канала межпроцессного взаимодействия. Один из которых служит для передачи информации от родительского процесса к дочернему процессу, а второй от дочернего процесса к родительскому процессу.

Родительский процесс запрашивает у пользователя название файла. Дочерний процесс перенаправляет свой стандартный ввод на чтение из родительского канала (используя `dup2`), тем самым получая название файла, а стандартный вывод на запись в родительский канал. Затем родительский процесс запускает программу считывает команды из стандартного потока ввода в формате "число<newline>", и передает их дочернему процессу. В это время дочерний процесс запускает программу `child.c`, которая проверяет число на простоту. Если число является натуральным составным дочерний процесс выводит число в файл и возвращает родительскому процессу значение "0", что сигнализирует о продолжении работы, в противном случае дочерний процесс возвращает родительскому процессу значение "1", что приводит к завершению родительского процесса. В программе предусмотрена функция остановки ввода "по желанию" с помощью ввода в родительский канал "q".

Код программы

main.c

```
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>

int main(int argc, char* argv[])
{
    if (argc != 1)
    {
        return 1;
    }

    char filename[256];
    const char msg[] = "Input the name of file: ";
    write(STDOUT_FILENO, msg, sizeof(msg));
    ssize_t bytes = read(STDIN_FILENO, filename, sizeof(filename) - 1);
    if (bytes <= 0)
    {
        const char error_msg[] = "Error: unable to get user data\n";
        write(STDERR_FILENO, error_msg, sizeof(error_msg));
        exit(EXIT_FAILURE);
    }
    filename[bytes-1] = '\0';

    int parent_to_child[2];
    if (pipe(parent_to_child) == -1)
    {
        const char error_msg[] = "Error: unable to create parent_to_child pipe\n";
        write(STDERR_FILENO, error_msg, sizeof(error_msg));
        exit(EXIT_FAILURE);
    }

    int child_to_parent[2];
    if (pipe(child_to_parent) == -1)
    {
        const char error_msg[] = "Error: unable to create child_to_parent pipe\n";
        write(STDERR_FILENO, error_msg, sizeof(error_msg));
        exit(EXIT_FAILURE);
    }

    pid_t child_id = fork();
    switch (child_id)
    {
        case -1:
            const char error_msg[] = "Error: unable to create child process\n";
            write(STDERR_FILENO, error_msg, sizeof(error_msg));
            exit(EXIT_FAILURE);
        case 0:
            close(parent_to_child[1]);
            dup2(parent_to_child[0], STDIN_FILENO);
            close(parent_to_child[0]);
```

```

close(child_to_parent[0]);
dup2(child_to_parent[1], STDOUT_FILENO);
close(child_to_parent[1]);

int exec_status = execl("./child", "child", NULL);
if (exec_status == -1)
{
    const char error_msg[] = "Error: unable to execl\n";
    write(STDERR_FILENO, error_msg, sizeof(error_msg));
    exit(EXIT_FAILURE);
}
default:
close(parent_to_child[0]);
close(child_to_parent[1]);

write(parent_to_child[1], filename, strlen(filename) + 1);

const char msg[] = "Input numbers (q for quit):\n";
write(STDOUT_FILENO, msg, sizeof(msg));

char buffer[128];
int bytes;
while ((bytes = read(STDIN_FILENO, buffer, sizeof(buffer) - 1)) > 0)
{
    buffer[bytes] = '\0';

    write(parent_to_child[1], buffer, bytes);

    char buffer2[128];
    ssize_t bytes2 = read(child_to_parent[0], buffer2, sizeof(buffer2) - 1);
    if (bytes2 <= 0)
    {
        const char error_msg[] = "Error: unable to read child info\n";
        write(STDERR_FILENO, error_msg, sizeof(error_msg));
        break;
    }
    buffer2[bytes2] = '\0';

    if (buffer2[0] == '1')
    {
        const char msg[] = "Done\n";
        write(STDERR_FILENO, msg, sizeof(msg));
        break;
    }
}

close(parent_to_child[1]);
close(child_to_parent[0]);

int exec_child_status;
wait(&exec_child_status);
if (WIFEXITED(exec_child_status))
{
    if (WEXITSTATUS(exec_child_status) != 0)
    {
        const char error_msg[] = "Error: child process terminated with error\n";

```

```

        write(STDERR_FILENO, error_msg, sizeof(error_msg));
        exit(EXIT_FAILURE);
    }
    exit(EXIT_SUCCESS);
}
else
{
    const char error_msg[] = "Error: the process is executing\n";
    write(STDERR_FILENO, error_msg, sizeof(error_msg));
    exit(EXIT_FAILURE);
}
}

return 0;
}

```

child.h

```

#ifndef CHILD_H
#define CHILD_H

typedef enum
{
    OK,
    NOT_OK
} status_code;

status_code check_positive_composite(const int number);

#endif

```

child.c

```

#include <unistd.h>
#include <stdlib.h>
#include <fcntl.h>
#include "../include/child.h"

status_code check_positive_composite(const int number)
{
    if (number == 1)
    {
        return OK;
    }

    for (int i = 2; i * i <= number; ++i)
    {
        if (number % i == 0)
        {
            return OK;
        }
    }

    return NOT_OK;
}

```

```

int main(int argc, char* argv[])
{
    if (argc != 1)
    {
        return 1;
    }

    char filename[256];
    ssize_t status = read(STDIN_FILENO, filename, sizeof(filename) - 1);
    if (status < 0)
    {
        const char error_msg[] = "Error: unable to read filename\n";
        write(STDERR_FILENO, error_msg, sizeof(error_msg));
        return 1;
    }
    filename[status] = '\0';

    int file = open(filename, O_CREAT | O_WRONLY | O_TRUNC, 0644);
    if (file == -1)
    {
        const char error_msg[] = "Error: unable to open file\n";
        write(STDERR_FILENO, error_msg, sizeof(error_msg));
        return 1;
    }

    char buffer[32];
    while (1)
    {
        ssize_t bytes = read(STDIN_FILENO, buffer, sizeof(buffer) - 1);
        if (bytes <= 0)
        {
            const char error_msg[] = "Error: unable to read info from parent\n";
        }
        buffer[bytes] = '\0';

        if (buffer[0] == 'q')
        {
            const char error_msg[] = "1";
            write(STDOUT_FILENO, error_msg, sizeof(error_msg) - 1);
            break;
        }

        char *endptr = NULL;
        int number = strtol(buffer, &endptr, 10);
        if (endptr == buffer || (*endptr != '\n' && *endptr != '\0'))
        {
            break;
        }

        status_code status = check_positive_composite(number);

        if (status == OK)
        {
            int i = 0;
            int num_copy = number;
            char number_buffer[32];
            do

```

```

    {
        number_buffer[i++] = num_copy % 10 + '0';
        num_copy /= 10;
    } while (num_copy > 0);

    int start = 0;
    int end = i - 1;
    while (start < end)
    {
        char temp = number_buffer[start];
        number_buffer[start] = number_buffer[end];
        number_buffer[end] = temp;
        ++start;
        --end;
    }
    number_buffer[i++] = '\n';

    write(file, number_buffer, i);

    const char msg[] = "0";
    write(STDOUT_FILENO, msg, sizeof(msg));
}
else
{
    const char error_msg[] = "1";
    write(STDOUT_FILENO, error_msg, sizeof(error_msg) - 1);
    break;
}
}

close(file);
return 0;
}

```

Протокол работы программы

Тестирование:

```
mrx@pc:~/Desktop/OS/Lab1/src$ gcc child.c -o child
mrx@pc:~/Desktop/OS/Lab1/src$ gcc main.c -o main
mrx@pc:~/Desktop/OS/Lab1/src$ ./main
Input the name of file: test.txt
Input numbers (q for quit):
10
4
5
Done
mrx@pc:~/Desktop/OS/Lab1/src$ ./main
Input the name of file: test2.txt
Input numbers (q for quit):
0
Done
mrx@pc:~/Desktop/OS/Lab1/src$ ./main
Input the name of file: test3.txt
Input numbers (q for quit):
10
14
26
102
-1
Done
mrx@pc:~/Desktop/OS/Lab1/src$ cat test.txt
10
4
mrx@pc:~/Desktop/OS/Lab1/src$ cat test2.txt
mrx@pc:~/Desktop/OS/Lab1/src$ cat test3.txt
10
14
26
102
```


Strace:

```
$ strace -f ./main
execve("./main", ["/main"], 0x7fffea5c8508 /* 60 vars */) = 0
brk(NULL)                               = 0x6251a9327000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x753e1a2f5000
access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=77459, ...}) = 0
mmap(NULL, 77459, PROT_READ, MAP_PRIVATE, 3, 0) = 0x753e1a2e2000
close(3)                                = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\3\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0"..., 832) = 832
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0@\0\0\0\0\0\0@\0\0\0\0\0\0"..., 784, 64) = 784
fstat(3, {st_mode=S_IFREG|0755, st_size=2125328, ...}) = 0
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0@\0\0\0\0\0\0@\0\0\0\0\0\0"..., 784, 64) = 784
mmap(NULL, 2170256, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x753e1a000000
mmap(0x753e1a028000, 1605632, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x753e1a028000
mmap(0x753e1a1b0000, 323584, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
0x1b0000) = 0x753e1a1b0000
mmap(0x753e1a1ff000, 24576, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1fe000) = 0x753e1a1ff000
mmap(0x753e1a205000, 52624, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x753e1a205000
close(3)                                = 0
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x753e1a2df000
arch_prctl(ARCH_SET_FS, 0x753e1a2df740) = 0
set_tid_address(0x753e1a2dfa10)        = 34037
set_robust_list(0x753e1a2dfa20, 24)    = 0
rseq(0x753e1a2e0060, 0x20, 0, 0x53053053) = 0
mprotect(0x753e1a1ff000, 16384, PROT_READ) = 0
mprotect(0x62518929c000, 4096, PROT_READ) = 0
mprotect(0x753e1a333000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
munmap(0x753e1a2e2000, 77459)          = 0
write(1, "Input the name of file: \0", 25Input the name of file: ) = 25
read(0, test.txt
"test.txt\n", 255)          = 9
pipe2([3, 4], 0)            = 0
pipe2([5, 6], 0)            = 0
clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD,
child_tidptr=0x753e1a2dfa10) = 34065
strace: Process 34065 attached
[pid 34037] close(3)              = 0
[pid 34065] set_robust_list(0x753e1a2dfa20, 24 <unfinished ...>
[pid 34037] close(6 <unfinished ...>
[pid 34065] <... set_robust_list resumed>) = 0
[pid 34037] <... close resumed>      = 0
[pid 34065] close(4 <unfinished ...>
[pid 34037] write(4, "test.txt\0", 9 <unfinished ...>
[pid 34065] <... close resumed>      = 0
[pid 34037] <... write resumed>      = 9
[pid 34065] dup2(3, 0 <unfinished ...>
[pid 34037] write(1, "Input numbers (q for quit):\n\0", 29 <unfinished ...>
```

```

[pid 34065] <... dup2 resumed>      = 0
[pid 34065] close(3)                = 0
Input numbers (q for quit):
[pid 34037] <... write resumed>     = 29
[pid 34065] close(5 <unfinished ...>
[pid 34037] read(0, <unfinished ...>
[pid 34065] <... close resumed>     = 0
[pid 34065] dup2(6, 1)              = 1
[pid 34065] close(6)                = 0
[pid 34065] execve("./child", ["child"], 0x7fff4674b658 /* 60 vars */) = 0
[pid 34065] brk(NULL)                = 0x582dbc705000
[pid 34065] mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1,
0) = 0x7bced6687000
[pid 34065] access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
[pid 34065] openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
[pid 34065] fstat(3, {st_mode=S_IFREG|0644, st_size=77459, ...}) = 0
[pid 34065] mmap(NULL, 77459, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7bced6674000
[pid 34065] close(3)                = 0
[pid 34065] openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
[pid 34065] read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0\0"..., 832) = 832
[pid 34065] pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784, 64) = 784
[pid 34065] fstat(3, {st_mode=S_IFREG|0755, st_size=2125328, ...}) = 0
[pid 34065] pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784, 64) = 784
[pid 34065] mmap(NULL, 2170256, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) =
0x7bced6400000
[pid 34065] mmap(0x7bced6428000, 1605632, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x7bced6428000
[pid 34065] mmap(0x7bced65b0000, 323584, PROT_READ,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1b0000) = 0x7bced65b0000
[pid 34065] mmap(0x7bced65ff000, 24576, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1fe000) = 0x7bced65ff000
[pid 34065] mmap(0x7bced6605000, 52624, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7bced6605000
[pid 34065] close(3)                = 0
[pid 34065] mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1,
0) = 0x7bced6671000
[pid 34065] arch_prctl(ARCH_SET_FS, 0x7bced6671740) = 0
[pid 34065] set_tid_address(0x7bced6671a10) = 34065
[pid 34065] set_robust_list(0x7bced6671a20, 24) = 0
[pid 34065] rseq(0x7bced6672060, 0x20, 0, 0x53053053) = 0
[pid 34065] mprotect(0x7bced65ff000, 16384, PROT_READ) = 0
[pid 34065] mprotect(0x582d8a8b5000, 4096, PROT_READ) = 0
[pid 34065] mprotect(0x7bced66c5000, 8192, PROT_READ) = 0
[pid 34065] prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) =
0
[pid 34065] munmap(0x7bced6674000, 77459) = 0
[pid 34065] read(0, "test.txt\0", 255) = 9
[pid 34065] openat(AT_FDCWD, "test.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 3
[pid 34065] read(0, 10
<unfinished ...>
[pid 34037] <... read resumed> "10\n", 127) = 3
[pid 34037] write(4, "10\n", 3)      = 3
[pid 34065] <... read resumed> "10\n", 31) = 3
[pid 34037] read(5, <unfinished ...>
[pid 34065] write(3, "10\n", 3)      = 3
[pid 34065] write(1, "\0", 2)       = 2
[pid 34037] <... read resumed> "\0", 127) = 2

```

```

[pid 34037] read(0, <unfinished ...>
[pid 34065] read(0, 14
<unfinished ...>
[pid 34037] <... read resumed>"14\n", 127) = 3
[pid 34037] write(4, "14\n", 3) = 3
[pid 34065] <... read resumed>"14\n", 31) = 3
[pid 34037] read(5, <unfinished ...>
[pid 34065] write(3, "14\n", 3) = 3
[pid 34065] write(1, "0\0", 2) = 2
[pid 34037] <... read resumed>"0\0", 127) = 2
[pid 34037] read(0, <unfinished ...>
[pid 34065] read(0, 26
<unfinished ...>
[pid 34037] <... read resumed>"26\n", 127) = 3
[pid 34037] write(4, "26\n", 3) = 3
[pid 34065] <... read resumed>"26\n", 31) = 3
[pid 34037] read(5, <unfinished ...>
[pid 34065] write(3, "26\n", 3) = 3
[pid 34065] write(1, "0\0", 2 <unfinished ...>
[pid 34037] <... read resumed>"0\0", 127) = 2
[pid 34065] <... write resumed>) = 2
[pid 34065] read(0, <unfinished ...>
[pid 34037] read(0, 102
"102\n", 127) = 4
[pid 34037] write(4, "102\n", 4) = 4
[pid 34065] <... read resumed>"102\n", 31) = 4
[pid 34037] read(5, <unfinished ...>
[pid 34065] write(3, "102\n", 4) = 4
[pid 34065] write(1, "0\0", 2) = 2
[pid 34037] <... read resumed>"0\0", 127) = 2
[pid 34037] read(0, <unfinished ...>
[pid 34065] read(0, -1
<unfinished ...>
[pid 34037] <... read resumed>"-1\n", 127) = 3
[pid 34037] write(4, "-1\n", 3) = 3
[pid 34065] <... read resumed>"-1\n", 31) = 3
[pid 34037] read(5, <unfinished ...>
[pid 34065] write(1, "1", 1) = 1
[pid 34037] <... read resumed>"1", 127) = 1
[pid 34037] write(2, "Done\n\0", 6 <unfinished ...>
[pid 34065] close(3)Done
<unfinished ...>
[pid 34037] <... write resumed>) = 6
[pid 34065] <... close resumed>) = 0
[pid 34037] close(4) = 0
[pid 34065] exit_group(0) = ?
[pid 34037] close(5) = 0
[pid 34065] +++ exited with 0 +++
--- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=34065, si_uid=1000, si_status=0,
si_utime=0, si_stime=0} ---
wait4(-1, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 34065
exit_group(0) = ?
+++ exited with 0 +++

```

Вывод

Во время выполнения лабораторной работы были изучены и использованы основные системные вызовы для работы с процессами и межпроцессным взаимодействием в Linux. Была создана программа, демонстрирующая создание процессов, создание каналов связи между ними и перенаправление стандартных потоков ввода-вывода.

В ходе выполнения возникли проблемы с корректной обработкой введенных данных, данная проблема разрешилась путем моей реализации перевода строк в числа и обратно, а так же добавлением дополнительной валидации ввода. Возникли сложности с правильным закрытием файловых дескрипторов для избежания утечек ресурсов.